



Politechnika Warszawska

**Wydział Matematyki
i Nauk Informatycznych**

Reprezentacja Wiedzy

Procesy działań złożonych

Instrukcja użytkownika

Zespół projektowy nr 4:

Wojciech Jasiński (koordynator)

Krzysztof Gryboś

Jarosław Bieniek

Filip Filipkowski

Sebastian Rydz

Jakub Pietrzak

Sandra Adamiec

Warszawa, 11 czerwca 2026

1 Uruchomienie

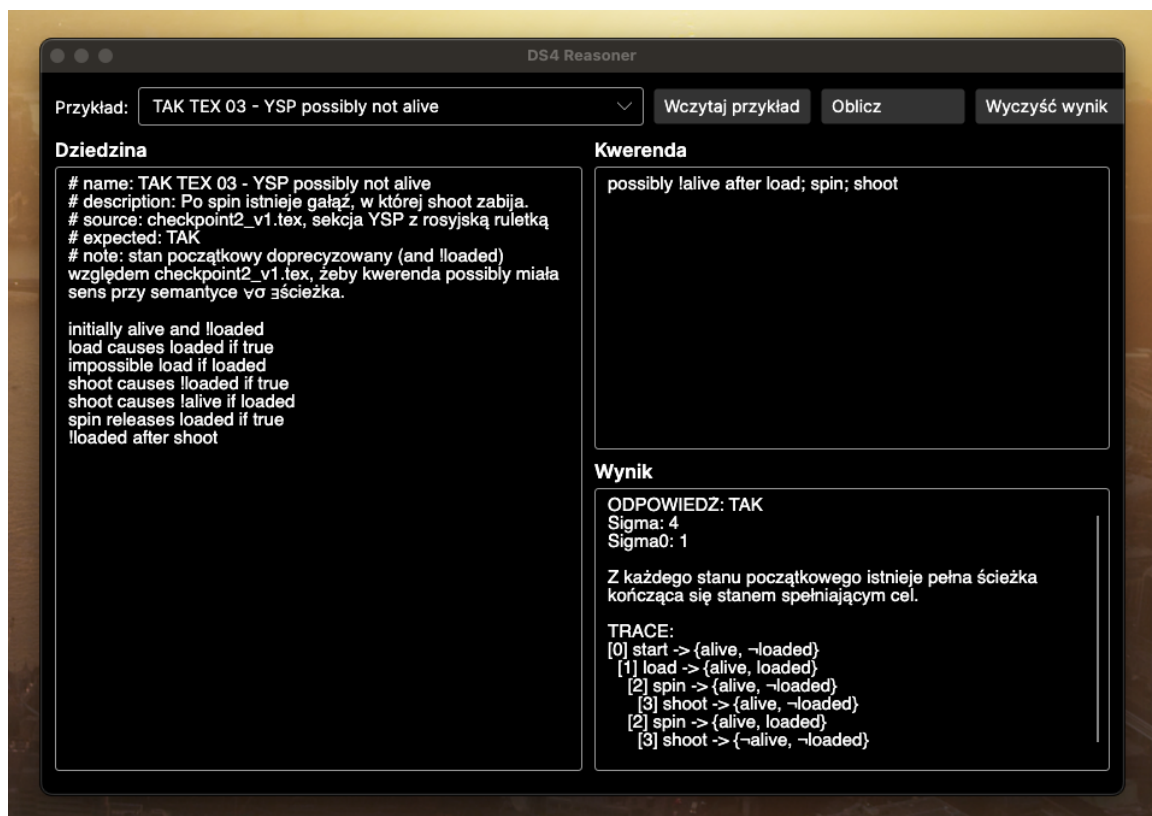
Gotowe pliki znajdują się w katalogu `app/release/`. Są samowystarczalne — nie wymagają instalowania platformy .NET.

- **Windows:** dwuklik na `Ds4.Gui.exe`.
- **Linux:** plik `Ds4.CrossGui` (w razie potrzeby `chmod +x Ds4.CrossGui`, potem `./Ds4.CrossGui`).

Katalog `examples/` musi pozostać obok pliku wykonywalnego — z niego program wczytuje listę wbudowanych przykładów.

Uruchomienie ze źródeł (wymaga .NET 8 SDK): w katalogu `app/` skrypt `RUN_GUI.bat` (Windows) albo `RUN_CROSS_GUI.sh` (Linux/macOS).

2 Okno programu



Okno ma cztery elementy:

1. **Pasek przykładów** (góra) — lista rozwijana z 57 wbudowanymi przypadkami oraz przyciski *Wczytaj przykład*, *Oblicz* i *Wyczyść wynik*.
2. **Dziedzina** (lewy panel) — edytowalny opis dziedziny w języku akcji (rozdział 3).
3. **Kwerenda** (prawy górny panel) — jedno pytanie do dziedziny (rozdział 4).
4. **Wynik** (prawy dolny panel) — odpowiedź wraz z uzasadnieniem i przebiegiem wykonania (rozdział 5).

Typowa praca: wybierz przykład i kliknij *Wczytaj przykład* (albo wpisz własną dziedzinę i kwerendę), następnie kliknij *Oblicz*.

3 Jak opisać dziedzinę

Dziedzinę opisuje się zdaniami — po jednym w wierszu. Wiersze zaczynające się od # to komentarze. Nazwy fluentów i akcji program zbiera sam z treści zdań (deklaracje fluents/actions są opcjonalne). Poniżej opis dziedziny „rosyjskiej ruletki” budowany krok po kroku.

Stan początkowy. Postać na początku żyje — o nabiciu broni nic nie mówimy, więc oba przypadki będą brane pod uwagę:

```
initially alive
```

Efekt akcji — `causes ... if ....` Strzał zabija, ale tylko gdy broń jest nabita:

```
shoot causes not alive if loaded
load causes loaded if not loaded
```

Efekt niedeterministyczny — `releases`. Zakręcenie bębniem sprawia, że `loaded` może wyjść dowolnie:

```
spin releases loaded if true
```

Niewykonalność — `impossible ... if ....` Nie można nabić broni już nabitej:

```
impossible load if loaded
```

Ograniczenie stałe — `always`. Gdyby w dziedzinie były fluenty powiązane (np. p zawsze pociąga q), zapisalibyśmy:

```
always p implies q
```

Takie ograniczenie działa we wszystkich stanach i wymusza skutki uboczne akcji.

W formułach (warunkach `if`, `initially`, `always`, celach kwerend) używa się wyłącznie operatorów słownych: `not` (nie), `and`, `or`, `implies`, `iff` oraz stałych `true/false`. Program nie przyjmuje symboli (`!`, `&`, `|`, `->`, `<->`) — zgłasza wtedy błąd.

4 Jak zadać kwerendę

Kwerenda to jedno zdanie. Pytamy o *proces*, czyli ciąg kroków rozdzielonych średnikami; w jednym kroku może wykonywać się równocześnie kilka akcji — zapisuje się je w nawiasach klamrowych:

```
spin; shoot           (dwa kroki po jednej akcji)
{lift_a, lift_b}; carry (dwie akcje naraz, potem jedna)
```

Cztery rodzaje pytań:

Zapis	Znaczenie
possibly executable after P	czy proces P <i>kiedykolwiek</i> da się wykonać do końca
necessary executable after P	czy proces P da się wykonać <i>zawsze</i>
possibly γ after P	czy P <i>kiedykolwiek</i> prowadzi do celu γ
necessary γ after P	czy P <i>zawsze</i> prowadzi do celu γ

„Zawsze” oznacza: dla każdego możliwego stanu początkowego i każdego przebiegu (przy niedeterminizmie); „kiedykolwiek” — że z każdego możliwego stanu początkowego istnieje przynajmniej jeden pełny przebieg. Przykład:

```
possibly not alive after spin; shoot
```

5 Jak czytać wynik

Panel *Wynik* zawiera kolejno:

- **ODPOWIEDŹ: TAK / NIE** — odpowiedź na kwerendę;
- **liczba stanów** — liczbę stanów dopuszczalnych $|\Sigma|$ (zgodnych z **always**) oraz liczbę możliwych stanów początkowych $|\Sigma_0|$;
- **uzasadnienie** — jedno–dwa zdania, np. wskazanie ścieżki (dla TAK przy **possibly**) albo kontrprzykładu (dla NIE przy **necessary**);
- **TRACE** — drzewo wykonań procesu. Każdy wiersz to jeden węzeł: numer kroku w nawiasach kwadratowych, wykonywana akcja i stan po niej (przedrostek **not** oznacza fluent fałszywy), np.

```
[0] start: {alive, not loaded}
  [1] spin: {alive, not loaded}
    [2] shoot: {alive, not loaded}
  [1] spin: {alive, loaded}
    [2] shoot: {not alive, loaded}
```

Wcięcie oznacza kolejny krok procesu; rozgałęzienie — kilka możliwych wyników (niedeterminizm). Wiersz **BLOCKED**: brak następników dla kolejnego kroku procesu oznacza, że w tym miejscu akcji nie dało się wykonać i ścieżka się urywa.

Sprzeczny opis. Jeżeli dziedzina jest sprzeczna, program nie pokazuje odpowiedzi TAK/NIE ani śladu, lecz komunikat o błędzie:

- sprzeczne ograniczenia **always** (żaden stan ich nie spełnia, $|\Sigma| = 0$) — **BŁĄD**: Model jest pusty: ograniczenia **always** są sprzeczne.;
- sprzeczny opis początkowy — gdy zdania **initially**, **after** lub **observable after** nie mogą zostać spełnione jednocześnie ($|\Sigma_0| = 0$) — **BŁĄD**: Brak stanów początkowych: **initially/after/observable after** są sprzeczne.

W obu przypadkach należy poprawić opis dziedziny.

6 Przykłady i praca z plikami

Wbudowane przykłady. Lista na pasku górnym zawiera 57 przypadków. Prefiks **tak_/nie_** mówi, jaka jest poprawna odpowiedź, a komentarze na początku dziedziny opisują, co dany przykład pokazuje.

Format plików. Każdy przykład to trzy pliki tekstowe o wspólnej nazwie:

- **<id>.domain** — opis dziedziny,
- **<id>.query** — jedna kwerenda,
- **<id>.expected** — oczekiwana odpowiedź (używana przez testy).

Własne przykłady wystarczy dograć do katalogu **examples/** — po ponownym uruchomieniu pojawią się na liście.

Wersja konsolowa. Te same obliczenia można wykonać bez GUI (wymaga .NET 8 SDK, katalog **app/**):

```
dotnet run --project src/Ds4.Cli -- plik.domain plik.query
```

Opcja **--examples** wypisuje listę wbudowanych przykładów, a **--example <id>** rozwiązuje wskazany przykład.